La capa de Transporte.

Principios del control de congestión.

Tema 3: La capa de Transporte

3.1 La capa de transporte y sus servicios

3.2 Multiplexación y demultiplexación

3.3 Transporte sin conexión: UDP

3.4 Principios de transferencia de datos fiable

3.5 Transporte orientado a conexión: TCP

3.6 Principios del control de congestión

- 3.6.1 Las causas y los costes de la congestión
- 3.6.2 Métodos para controlar la congestión

3.7 Mecanismo de control de congestión de TCP

Principios de control de la congestión

- Congestión de la red.
  - Consecuencia del aumento del tráfico en la red.
    - Primero se manifiesta con el aumento del retardo.
    - Luego, cuando se hace más severa, desbordamiento de los *buffers* de los *routers*.
      - Descarte/pérdida de paquetes en los *routers* ➔ TCP responde retransmitiendo.
      - Reducción del rendimiento ofrecido a las aplicaciones.
  - Retransmisión de paquetes (mecanismo de fiabilidad).
    - Se ocupa de un síntoma de la congestión.
      - "A veces puede ser peor el remedio que la enfermedad".
    - NO se ocupa de la causa de la congestión.
      - Muchos emisores generando tráfico a velocidades mayores que las recomendables en situaciones concretas.

Consecuencias de la congestión

- Escenario 1

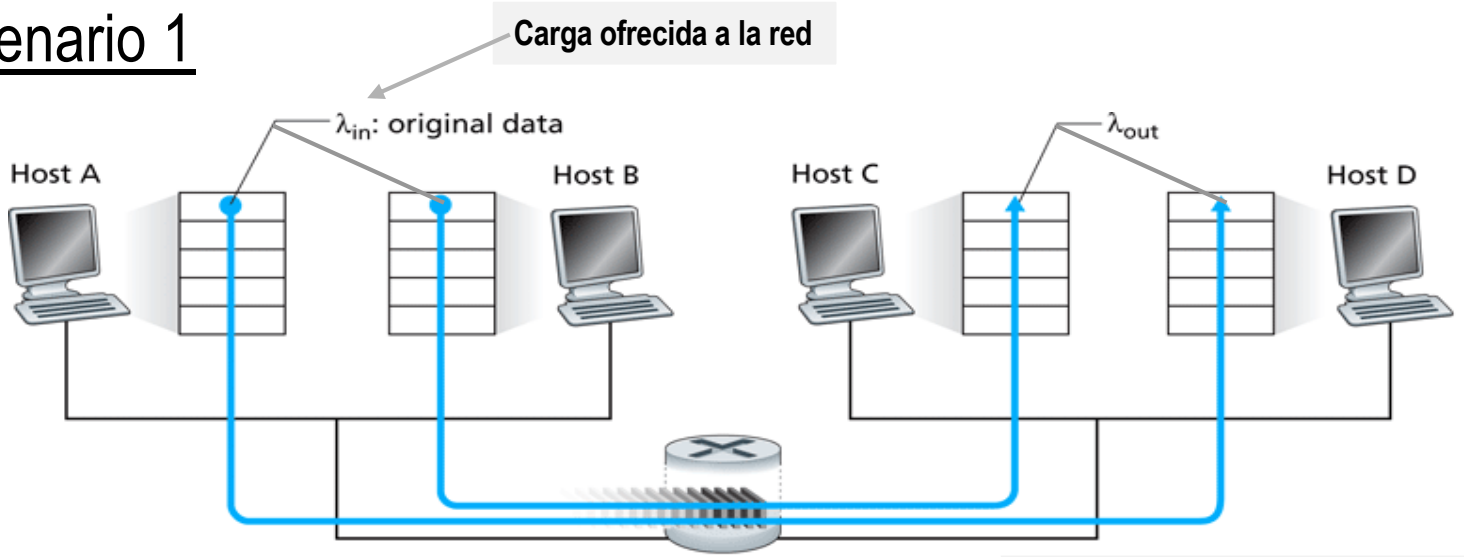


Figure 3.43 ♦ Congestion scenario 1: Two connections sharing a single hop with infinite buffers

- Sólo datos originales.
- Protocolo de Transporte simple:
  - NO control de errores (NO retransmisiones), NO control de flujo, NO control de congestión.
- Se ignora sobrecarga de cabeceras → λ_{in} Bps = velocidad con que las aplicaciones entregan datos al *router.*
- Capacidad del enlace de salida del *router* compartida entre los *hosts* A y B.

Consecuencias de la congestión

- Escenario 1

Rendimiento del Escenario 1

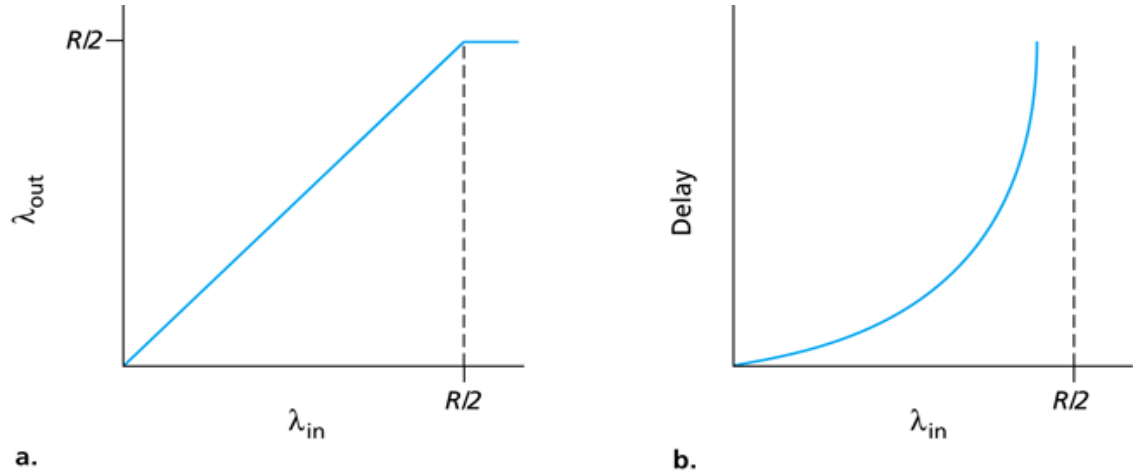


Figure 3.44 ♦ Congestion scenario 1: Throughput and delay as a function of host sending rate

Operar a una tasa de transferencia agregada ≅ R puede parecer ideal desde el punto de vista de λ_{out} , pero…

⬇

Grandes retardos de cola cuando la tasa de llegada de paquetes se aproxima a la capacidad del enlace R.

Tasa de transferencia por conexión

- $\lambda_{out} = f(\lambda_{in})$
- Si $\lambda_{in} < R/2$ ➔ $\lambda_{out} = \lambda_{in}$
  - Receptor recibe todo lo que envía el emisor
- Si $\lambda_{in} > R/2$ ➔ $\lambda_{out} = R/2$
  - Resultado de la compartición de la capacidad del enlace
- Hosts C y D NUNCA dispondrán de una $\lambda_{out} > R/2$

Retardo por conexión

- Retardo = f (λ_{in})
- λ_{in} → R/2 ➔ Retardo crece considerablemente
- Si $\lambda_{in} > R/2$ ➔ Retardo infinito
  - Suponiendo *buffers* de capacidad INFINITA y transmitiendo durante un período de tiempo INFINITO
- Operar a una tasa agregada próxima a R ➔ Retardos INFINITOS

Consecuencias de la congestión

- Escenario 2

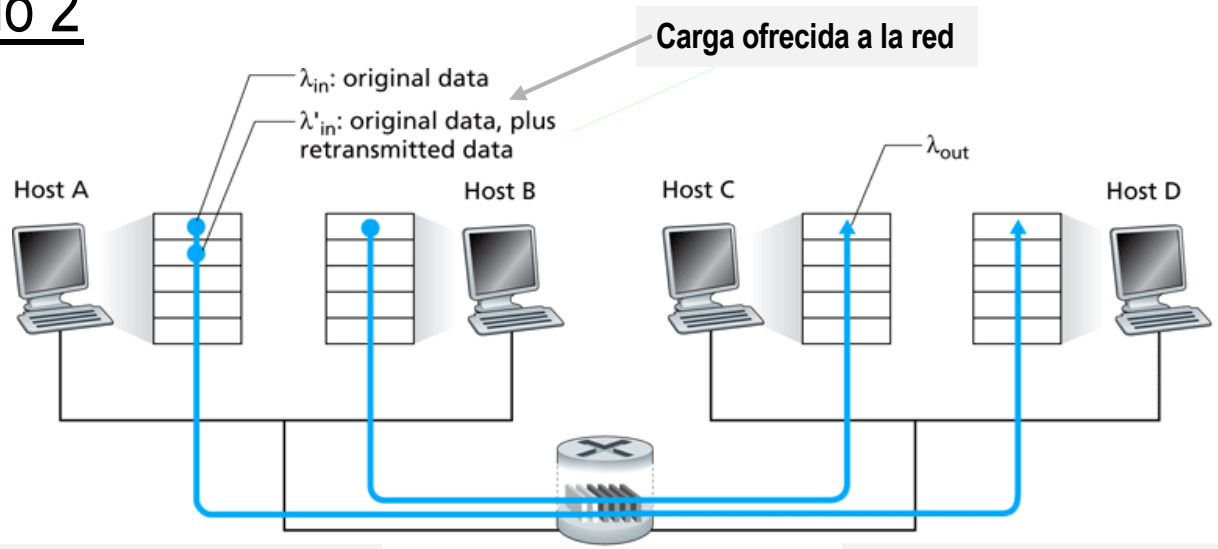


Figure 3.45 ♦ Scenario 2: Two hosts (with retransmissions) and a router with finite buffers

- Datos originales y retransmisiones.
- *Buffer* FINITO ➔ Descarte de paquetes.
- Protocolo de transporte FIABLE, gestión de errores con retransmisiones ➔ **Rendimiento según cómo se hagan las retransmisiones**
- λ_{in} = Velocidad a la que la aplicación envía datos originales al *socket* (en Bps).
- λ'_{in} = Velocidad a la que la capa de transporte envía segmentos a la red (originales y retransmitidos, en Bps).
- Capacidad del enlace de salida del *router* compartida entre los *hosts* A y B.

Consecuencias de la congestión

Escenario 2

Rendimiento del Escenario 2

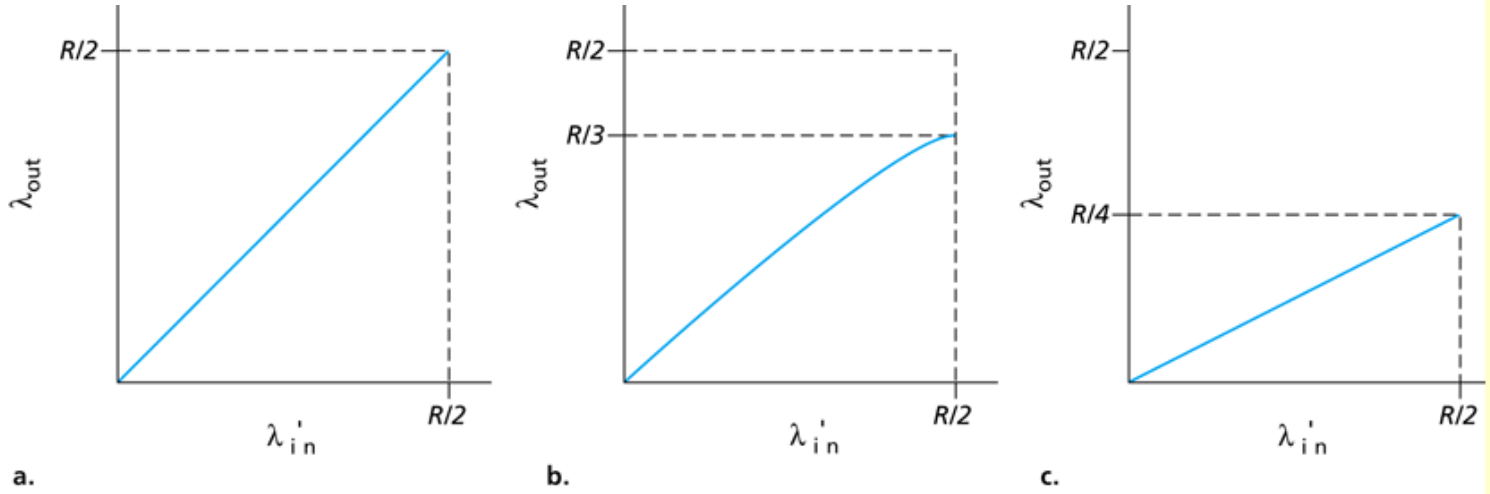


Figure 3.46 ♦ Scenario 2 performance with finite buffers

Caso c), más realista aún que el caso b)

- Si emisor retransmite cuando el *timeout* expira y el paquete NO se ha perdido, sino sólo demorado ➔ **Retransmisión PREMATURA**
- Paquete original y duplicado llegan al receptor ➔ Descarte del duplicado
  - **Se desaprovechada la capacidad de transmisión del *router*.**
  - **Si el *router* envía, en media, cada paquete dos veces:**
    - λ_{out} → R/4 cuando λ_{in} → R/2

Caso a), NO realista

- *Host* A conoce el estado del *buffer* (lleno o vacío). ➔ Envía datos SÓLO cuando el *buffer* está vacío.
- No se producen pérdidas de paquetes (**no retransmisión**).
- $\lambda_{in} = \lambda'_{in} = \lambda_{out}$ (tasa de transferencia) ➔ Rendimiento IDEAL (nada se pierde)
- $\lambda_{in} \leq R/2$, ya que NO se pierden paquetes.

Caso b), algo más realista que el caso a)

- Emisor **retransmite** SÓLO cuando tiene CERTEZA que un paquete se ha perdido (p.e, *Timeout* muy grande)
- Si $\lambda'_{in} = R/2$ ➔ $\lambda_{out} = R/3$ ➔ De R/2 unidades de datos enviadas, en media:
  - 0,333·R son datos originales (≅ 66,6 %)
  - **0,166· R son datos retransmitidos (≅ 33,3 %)**

Consecuencias de la congestión: resumen

- Costes debidos a la congestión de la red.
  - Grandes retardos de cola.
    - Cuando la tasa de llegada de paquetes se aproxima a la capacidad del enlace (caso 1).
  - Retransmisiones.
    - Emisor tiene que retransmitir para compensar la pérdida de paquetes por *buffers* desbordados (casos 2 y 3).
  - Desperdicio de recursos (ancho de banda y capacidad en los *buffers*)
    - Debido a retransmisiones innecesarias por grandes retardos (casos 2 y 3), reenvío de copias innecesarias.
    - Debido fundamentalmente al descarte de paquetes "avanzados" en una ruta (caso 3).

- La congestión de la red → Reducción del rendimiento que perciben las aplicaciones.

Métodos de control de congestión

- En general las técnicas para control de congestión se pueden clasificar atendiendo a dos casos:
  - La capa de red NO interviene.
    - Control de congestión "terminal a terminal".
  - La capa de red SÍ interviene.
    - Control de congestión "asistido por la red".

- Control de congestión "terminal a terminal".
  - La capa de red no provee soporte explícito a la capa de transporte(p.e, caso de TCP).
  - La congestión tiene que ser inferida por los *hosts* a partir sólo del comportamiento observado de la red, p.e:
    - Pérdida de segmentos TCP.
      - Por vencimiento del *timeout*, tres ACKs duplicados.
    - Retardos crecientes.